SCL-SLAM: A Scan Context-enabled LiDAR SLAM Using Factor Graph-Based Optimization

Zhiqiang Chen
School of System Science and Engineering
Sun Yat-Sen University
Guangzhou, China
chenzhq56@mail2.sysu.edu.cn

Yuhua Qi*
School of System Science and Engineering
Sun Yat-Sen University
Guangzhou, China
qiyh8@mail.sysu.edu.cn

Shipeng Zhong
School of System Science and Engineering
Sun Yat-Sen University
Guangzhou, China
zhongsp5@mail2.sysu.edu.cn

Dapeng Feng
School of System Science and Engineering
Sun Yat-Sen University
Guangzhou, China
fengdp5@mail2.sysu.edu.cn

Qiming Chen
College of Material and Energy
South China Agricultural University
Guangzhou, China
597888108@qq.com

Hongbo Chen
School of System Science and Engineering
Sun Yat-Sen University
Guangzhou, China
chenhongbo@mail.sysu.edu.cn

***Abstract*—In this paper, we present a complete LiDAR SLAM framework, SCL-SLAM, by integrating the loop closure module with the Scan Context descriptor into the tightly-coupled LiDAR-Inertial odometry FAST-LIO2. As a front-end, the direct LiDAR-Inertial odometry module efficiently and robustly produces motion estimates and undistorted scans. Toward the global localization based on 3D LiDAR scans, the lightweight Scan Context descriptor is used in the loop detection module. Additionally, the scan input is filtered through the keyframe selection module to improve the computation efficiency. As a back-end, a pose graph optimization is performed for the optimized trajectory and globally consistent map. SCL-SLAM is extensively evaluated on public datasets and a robot platform over various scales and environments. Experimental result shows that SCL-SLAM achieves higher accuracy than other state-of-art LiDAR SLAM systems and real-time performance. We also extend the proposed system to a centralized architecture SLAM framework for the robot team to use with 3D LiDAR observations.**

***Keywords*—LiDAR SLAM, Loop Closure, Multi-robot SLAM**

I. Introduction

LiDAR Simultaneous Localization and Mapping (SLAM) is a fundamental building block for many robotic applications, such as self-driving, map reconstruction, and warehouse inspection. Thanks to the ability to build an accurate map of an unknown environment and simultaneously localize itself in real time, SLAM can estimate the robot's state and provide information about the environment for path planning and obstacle avoidance. Vision-based SLAM is commonly used on mobile robots due to their low cost and lightweight. However, feature-based visual SLAM [1] is sensitive to texture and light and can only provide sparse feature map, whereas direct visual SLAM [2] requires photometric calibration and is computationally expensive, RGB-D based visual SLAM [3] work only indoor and sensitive to illumination change. LiDAR SLAM could overcome these difficulties by providing high-fidelity 3D measurements and delivering point clouds for scan registration to achieve an accurate pose estimation for an acceptable computational complexity [4].

Many LiDAR SLAM systems were proposed decades ago. LOAM (LiDAR odometry and mapping) framework [5] is one of the most successful methods, and it can be divided into three modules: feature extraction, LiDAR odometry, and mapping. After that, many LiDAR odometry (LO) variants of LOAM like LeGo-LOAM [6] have been developed to improve LOAM's robustness and computational expense. LOAM-Livox [7] is the variant of LOAM framework adopted to small FoV solid-state LiDARs. Moreover, some systems like LIO-SAM [8] and LiLi-OM [9] incorporate IMU data to significantly improve the estimation accuracy of the LiDAR odometry by using IMU information for point cloud de-skew and an excellent initial pose required by ICP (Iterative Closest Point). Recently, direct SLAM methods have emerged as the primary trend in SLAM developments due to the ability to achieve fast pose estimation by eliminating feature extraction. Direct methods, such as D-LIOM [10], DLO [11] and FAST-LIO2 [12], directly use LiDAR scan points to estimate robot pose by minimizing the energy function using point-to-plane residuals. FAST-LIO2 is the state-of-the-art direct LiDAR-Inertial SLAM system, but this method does not cooperate with a loop closure module to eliminate accumulated error.

Loop detection is generally associated with place recognition to reduce localization drift, playing an essential role in a complete SLAM system. The place recognition methods for LiDAR SLAM can be categorized into two kinds: local loop closure descriptors and global loop closure descriptors. Specifically, the LiDAR scan can be encoded into a descriptor, which is supposed that there is a loop if the current descriptor is matched with another descriptor in the database. Local descriptors, like spin image [13], PFH [14], shape context [15], or SHOT [16] have limitations since they are originally designed for point cloud matching not for place recognition. Global descriptors, such as M2DP [17], SegMatch [18] and Scan Context [19], are more suitable for place recognition in most scenarios than local descriptors. M2DP [17] projects LiDAR point cloud to multiple 2D images and describes the scan using a 192-dimensional global representation. SegMatch [18] is a segment-based matching algorithm requiring all points represented in a global reference frame. Scan Context is a lightweight histogram-based global descriptor proposed by [19], which performs better than the above LiDAR global descriptors and is robust to noise and point cloud density changes.

In this paper, we propose a complete LiDAR SLAM

*Corresponding author

This work was supported by the project "Development of sensing data collection and experimental platform for unmanned swarm system" from AMOVLAB.

2022 IEEE International Conference on Unmanned Systems (ICUS) | 978-1-6654-8456-5/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICUS55513.2022.9987005

978-1-6654-8456-5/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on February 28,2023 at 09:52:19 UTC from IEEE Xplore. Restrictions apply.

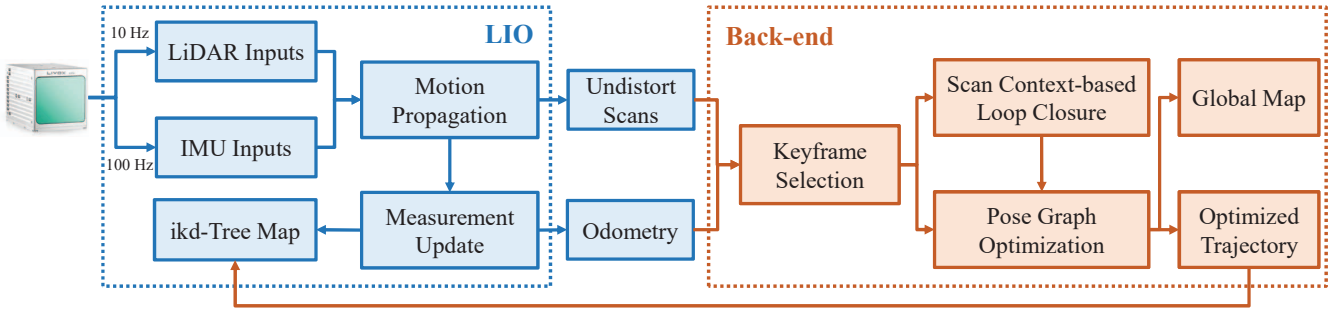


Fig. 1. System overview of SCL-SLAM consists of two parts: LiDAR-Inertial odometry and pose graph back-end. LIO produces undistorted scans by the motion propagation and estimates the LiDAR state by scan-to-map registration. Back-end detects loop closure using the Scan Context descriptor and incrementally adds the LIO factors and loop closure factors to get optimized trajectory and reconstruct the global map when necessary.

framework, called SCL-SLAM, by integrating the tightly-coupled LiDAR-Inertial odometry FAST-LIO2 and the loop closure module with the Scan Context descriptor to deliver superior odometry accuracy and more consistent map. Using IMU and LiDAR data, FAST-LIO2 produces the odometry and undistorted scans through direct scan to local map registration. The undistorted scans and odometry are fed into a designed keyframe selection module to achieve better real-time performance. Finally, using the results of keyframe selection, a loop detector using Scan Context descriptors and a pose graph optimization having multi constraints are performed for the optimized trajectory and globally consistent map. In summary, we make the main contributions as follows:

- We develop a LiDAR SLAM framework called SCL-SLAM by integrating the loop closure module with Scan Context descriptor into the LiDAR-inertial odometry FAST-LIO2, improving the accuracy and global consistency of LIO.
- A keyframe selection module is adapted to balance computing consumption and accuracy. It maintains a pose graph with sparse density and low memory consumption, making the nonlinear optimization efficient and preventing the failure of pose graph optimization with long-distance and long-time data.
- Extensive experiments are performed on various datasets sequences and outdoor scenarios to validate the proposed system. Moreover, we extend our framework to the multi-robot scenario to construct a global map cooperatively.

In the following sections, we detail our method's pipeline and each key module in Section II. Then, Section III gives the benchmark comparison on open datasets, real-world scenario tests, and multi-robot scenario tests. Finally, the conclusion is put in Section IV.

II. METHODOLOGY

A. System Overview

The pipeline of SCL-SLAM is shown in Fig.1. The LiDAR (e.g., Livox Avia) streams out LiDAR scans of 10 Hz and synchronized IMU data of 100 Hz. SCL-SLAM estimates the ego-motion of the robot and obtains a globally consistent map simultaneously. Once the system receives IMU and LiDAR data, the LIO module is activated for fast motion estimation and producing undistorted scans. These two outputs are then fed into the keyframe selection module. The Scan Context-based loop detector uses the selected undistorted scans to construct Scan Context descriptors for describing and matching features. After pose graph construction with LIO factors and loop closure factors, we optimize the pose variables with pose graph optimization. Finally, our system uses refined odometry and keyframe scans to maintain a consistent map.

B. LiDAR-Inertial Odometry

Our LiDAR-Inertial odometry includes two parts:

1) Motion propagation: Since the measurements of a LiDAR scan are not collected simultaneously, LiDAR scans always suffer from motion distortion. SCL-SLAM solves this problem using IMU information to model the in-scan motion. First, LIO uses forward propagation based on the IMU measurements to predict motion states. Then, backward propagation is used to estimate the pose of each point in the scan relative to the pose at the scan end time by fusing IMU measurements. Finally, each LiDAR point is transformed with these poses into the frame at the end of scanning time for motion distortion compensation.

2) Measurement update: An error-state iterated Kalman filter is used to update the motion measurement. In each iteration, the undistorted scan is matched with the global LiDAR map by finding a set of point-to-plane correspondences. Then, based on the updated state, the LIO module matches the current frame points with the map points, and the sensors' motion is iteratively refined by minimizing the energy function using point-to-plane residuals. Finally, if the state converges, the state of the last iteration is token as the posterior estimate.

With the estimated motion, the current LiDAR scan is transformed into the global coordinate system and incorporated with the global map. The LiDAR-Inertial odometry also proposes the ikd-Tree data structure, enabling incremental point insertion, deletion, and dynamic rebalancing for the local map. More details about the LIO module can be found in [12].

C. Keyframe Selection

Keyframe selection enables the real-time performance of back-end optimization since adding every LiDAR frame as a factor to the pose graph is computationally intractable. Our keyframe selection strategy is as follows:

- Compared with the latest keyframe, the frame with a distance change greater than τ is selected as the keyframe ($\tau = 1$ m for SCL-SLAM Evaluation).

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on February 28,2023 at 09:52:19 UTC from IEEE Xplore. Restrictions apply.

- Compared with the latest keyframe, the frame with a rotation angle change greater than θ is selected as the keyframe ($\theta = 10^\circ$ for SCL-SLAM Evaluation).
- Compared with the latest keyframe, the frame with a time difference of more than a certain number of regular frames is then selected as a new keyframe (five for SCL-SLAM Evaluation).

As long as one of the above conditions is met, this scan will be determined as a keyframe, and the LiDAR frames between two keyframes are only used for LIO motion measurement update and not added to the pose graph for global optimization. Apart from achieving a balance between map quality and memory consumption, adding keyframes under these criteria also helps maintain a relatively sparse pose graph, which makes the nonlinear optimization more time-efficient.

D. Scan Context Feature Description and Matching

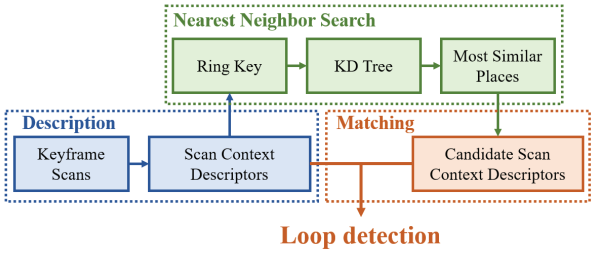


Fig. 2. Scan Context feature description and matching module.

The overall pipeline of Scan Context-based loop closure module is depicted in Fig.2, including description, nearest neighbor search, and descriptors matching. The description part of Scan Context describes the keyframe by projecting all points onto an image, where the intensity of image points is set by the maximum height of corresponding 3D points. The nearest neighbor search part of Scan Context is a two-step search process, fusing pairwise scoring and nearest search hierarchically to achieve a feasible searching time. First, the 2D plane image is divided into grid cells according to a specified number of sectors N_s and a ring key feature of dimension N_r. And then, the number of rings N_r is for building the KD tree to retrieve the nearest candidates. Finally, the matching module compares the retrieved candidates to the query. There is a loop when the candidate meets the acceptance threshold and is closest to the query. Once the loop closure is detected, we perform an ICP between the current LiDAR scan and the candidate local map to create a loop constraint (i.e., the relative translation and orientation between the two frames) if the ICP fitting score is smaller than a threshold (0.2 in our experiments).

E. Pose Graph Construction and Optimization

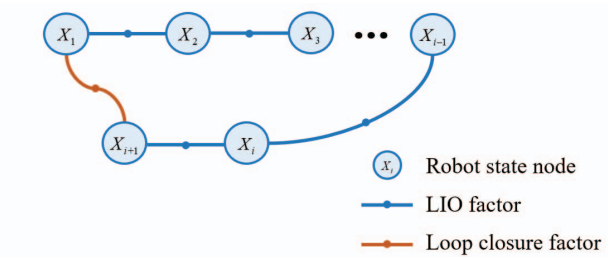


Fig. 3. Pose Graph construction with two types of factors.

Pose graph construction includes LIO factors and loop closure factors corresponding to the front-end and the loop closure module separately, along with robot state nodes, shown in Fig.3. The robot state node represents the robot's pose for a specific keyframe.

- LIO factors are the processes of initializing new robot state nodes and connecting them with their previous consecutive nodes. These sequential constraints are conducted by LIO updated pose and filter by keyframe selection module.
- Loop closure factors create a non-local pose constraint between two nodes X_j and X_k that are separated in time. Successful loop detection triggers a global pose optimization to maintain a globally consistent map.

Pose graph optimization aims to find the optimal solution with LIO and loop closure constraints. It can be formulated as a maximum a posteriori (MAP) problem, which is equivalent to minimizing a sum of nonlinear least-square errors as follows:

$$X^{\mathrm{MAP}} = \underset{X}{\operatorname{argmin}} \sum_{i} \left\| h\left(X_i, X_{i+1}\right) - z_i \right\|^2_{\Sigma_{o_i}} + \sum_{(j,k)} \left\| s_{jk}\left(h\left(X_j, X_k\right) - z_{jk}\right) \right\|^2_{\Sigma_{l_{jk}}},$$

where i is the index of each node. j and k are indexes of a Scan Context-based loop pair. s_{jk} is a scaling factor [20] for the loop closure factors between the node j and k. $h(\cdot)$ is a relative transformation between the two-argument nodes. z_i and z_{jk} are LIO measurement and loop closure measurement. Covariance matrices for odometry and loop constraints Σ_o and Σ_l determine the uncertainties of each measurement.

F. Map Reconstruction

Our LIO module registers raw points of a new scan to an incrementally-built point cloud map and calculates the robot's pose by minimizing the energy function using point-to-plane residuals. However, since each scan should be registered in the global coordinate system to construct a local map, the map will drift due to the cumulative pose error. Specifically, previous odometer errors reduce the precision of the constructed map, which in turn reduces the accuracy of subsequent motion estimation. We reconstruct the map according to the pose optimization result to address this issue. Our map reconstruction includes local map (ikd-Tree map [21]) reconstruction and global map reconstruction. Local map reconstruction ensures the accuracy of long-term direct scan-to-map registration by eliminating accumulated error causal state estimator. Global map reconstruction maintains a globally consistent reconstruction of the real-world environment. The map reconstruction is performed when a place is revisited.

III. EXPERIMENTS AND RESULTS

A. Implementation and Evaluation Setup

We implement SCL-SLAM in C++ using ROS. All modules of SCL-SLAM shown in Fig.1 are structured as only one node with multi-threads. We perform all optimization integrated with GTSAM [22] for rectifying the global pose graph at loop closures and use an iSAM-based [23] solvers to solve the MAP problem above.

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on February 28,2023 at 09:52:19 UTC from IEEE Xplore. Restrictions apply.



(a) The mobile platform.

(b) Trajectories visualization of SYSU Dataset in the satellite image with different colors.

Fig. 4. Setup for SYSU dataset collection.

We compare the proposed framework with LiLi-OM [9], FAST-LIO2 [12] and LIO-Livox[1] in terms of accuracy on 6 sequences of the public dataset and 9 sequences of our collected dataset. All methods are tested for solid-state LiDAR and run on a laptop equipped with a 12th Gen Intel i7-12700F CPU in Ubuntu.

1) Public dataset: The public dataset *lili* from the work LiLi-OM, is collected by a sensor suite composed of solid-state 3D Livox Horizon LiDAR sampled at 10 Hz, an Xsens MTi-670 IMU and a built-in IMU model BMI088 of Livox Horizon both sampled at 200 Hz. The *FR-IOSB* sequences were recorded at the Fraunhofer IOSB campus, and the *KA-Urban* sequences were recorded in urban scenarios.

2) SYSU dataset: To verify our algorithm in various real-world scenarios, the *SYSU dataset* was explicitly gathered for this study at Sun Yat-Sen University (East Campus), using a Scout mini mobile platform shown in Fig.4a. Our platform is composed of a Livox Avia LiDAR sampled at 10 Hz, an internal IMU of Avia sampled at 100 Hz, and an RTK GNSS (Global Navigation Satellite System) receiver recorded at 1 Hz for ground truth comparison. The built-in IMU is synchronized with the Avia LiDAR. Moreover, we use the PTP protocol to synchronize the RTK data and LiDAR frames. We collect 9 sequences of data in different scenes, including building scenarios (*Laboratory Building*, *Teaching Building*, *JIE Building*, *Arts Building*), campus road scenarios (*Campus-Road-1*, *Campus-Road-2*, *Campus-Road-3*, *Playground*) and a sequence *Tunnel* contains a path through the tunnel. The details of SYSU dataset are shown in Table I and Fig.4b.

TABLE I. SYSU DATASET DETAILS

Sequence	Scans	Trajectory length (m)	Duration (s)
Teaching-Building	8943	1184	894
Laboratory-Building	12010	1796	1201
JIE-Building	7596	989	759
Arts-Building	4420	523	442
Campus-Road-1	11170	1541	1117
Campus-Road-2	7018	950	702
Campus-Road-3	5033	698	503
Playground	8957	1113	895
Tunnel	7180	937	718

[1]https://github.com/Livox-SDK/LIO-Livox

B. Performance Metrics

Absolute trajectory error (ATE) of the robot pose concerning the provided ground truth is an error metric. Here we use the calculation method mentioned in [24], the trajectories should be aligned with the ground truth on the same coordinate firstly. And then we have the absolute trajectory error matrix as follows:

$$E_i := Q_i^{-1} S P_i,$$

where S is the alignment rigid-body transformation, P_i is the estimated pose at time i, and Q_i is the ground truth pose at time i. ATE measures the error between estimated and ground truth trajectory, defined as:

$$\mathrm{ATE}_{rmse} = \left(\frac{1}{n} \sum_{i=1}^{n} \left\| \mathrm{trans}\left(E_i\right) \right\|^2 \right)^{\frac{1}{2}}.$$

C. Accuracy Evaluation

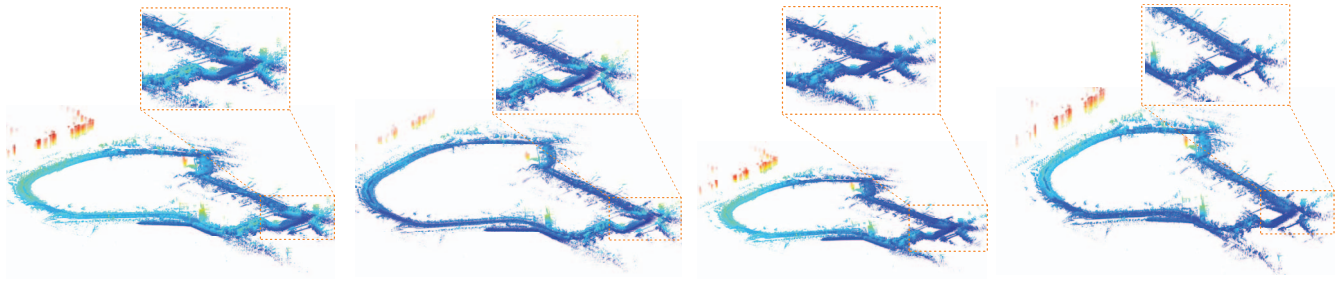
1) Public dataset experiment: The comparisons of the pose error from our method, three mainstream state-of-art SLAM framework evaluated on *lili* dataset shown Table II.

TABLE II. ABSOLUTE TRANSLATION ERROR (RMSE, METERS) ON *lili* DATASET

	FAST-LIO	LiLi-OM	LIO-Livox	Ours
KA-Urban-Campus-1	3.63	3.65	3.62	**3.58**
KA-Urban-Campus-2	4.08	4.06	**3.99**	4.08
KA-Urban-East	15.54	13.69	19.57	**12.17**
KA-Urban-Schloss-1	2.6	**2.15**	2.21	2.19
FR-IOSB-Long	5.07	**3.81**	4.46	4.01
FR-IOSB-Short	4.46	4.26	4.37	**4.21**

Thanks to the accurate loop closure performance, the pose estimation of SCL-SLAM is very close to the ground truth and achieves similar or better performance to other state-of-the-art methods. Compared with other LiDAR-Inertial SLAM methods, ours achieves the best performance in half sequences in *lili*, and the other three sequences achieve similar accuracy to the best. LIO-Livox achieves the best performance of sequence *KA-Urban-Campus-2*, but cannot keep it, especially on sequence *KA-Urban-East*. FAST-LIO2 and LIO-Livox show lousy performance on some long-time and long-distance sequences like *FR-IOSB-Long* and *KA-Urban-East* because their motion estimation relies solely

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on February 28,2023 at 09:52:19 UTC from IEEE Xplore. Restrictions apply.



(a) FAST-LIO2 (b) LiLi-OM (c) LIO-Livox (d) Ours

Fig. 5. Mapping results of different algorithms using sequence *Laboratory-Building*.

on the LIO and can not eliminate accumulated pose drift as the lack of loop correction module to estimate the relative poses of loop closures and optimize the poses globally. By using different tuned parameters for each sequence and its loop closure method, LiLi-OM achieves the best results for sequences *FR-IOSB-Long* and *KA-Urban-Schloss-1*, but our method still achieves similar performance to LiLi-OM.

2) SYSU dataset experiment: Table III shows the RMSE results of nine sequences we collected. Our method shows superior tracking accuracy (bold) in the most scenario and outperforms all other methods clearly in terms of RMSE in six of the total nine sequences. On the other hand, FAST-LIO and LIO-Livox show less accuracy with the long-time and long-distance sequences, such as *Playground*, *Campus-Road-1*, *Laboratory Building*, especially LIO-Livox lose its declared robustness and shows bad performance on *Tunnel*. Thanks to loop closure ability, LiLi-OM shows good performance in three sequences *JIE Building*, *Tunnel* and *Campus-Road-2* but can not keep it on other sequences, such as *Campus-Road-1*, *Laboratory Building* and *Playground*. Generally, our method maintains good results in various scenarios, even on sequences with loop closure like *Campus-Road-3*.

TABLE III. ABSOLUTE TRANSLATION ERROR (RMSE, METERS) ON REAL-WORLD SCEANRIOS

	FAST-LIO	LiLi-OM	LIO-Livox	Ours
Teaching-Building	4.50	4.53	4.60	**4.35**
Laboratory-Building	4.57	5.04	4.38	**2.86**
JIE-Building	2.45	**2.16**	2.39	2.21
Arts-Building	2.19	1.78	1.78	**1.71**
Campus-Road-1	4.28	4.11	3.69	**2.82**
Campus-Road-2	3.81	**3.68**	3.86	3.75
Campus-Road-3	2.81	2.77	2.93	**2.73**
Playground	3.34	4.04	3.29	**2.99**
Tunnel	2.38	**2.15**	18.78	2.36

We also show the results of the map construction on sequence *Laboratory Building* in Fig.5, where our method produces a more consistent map using the proposed loop closure scheme, while all other algorithms reconstruct the maps drift a lot at the z coordinate. The results of all the sequences in the other three methods show that they all have large deviations on the z-axis, even if LiLi-OM has a loop closure detection module, while our method maintains consistency and is robust enough to achieve good results in most tests.

D. Processing Time Evaluation

To verify the real-time performance of SCL-SLAM, we evaluate the computation efficiency of our keyframe-based system (SCL-SLAM (w/ keyframe)) and compare it with our complete system without the keyframe selection module (SCL-SLAM (w/o keyframe)). The processing time on the sequence *Playground* with 895.97 s duration is shown in Table IV.

It should be noted that we only calculate the processing time of the LIO module time per scan since both SCL-SLAM (w/ keyframe) and SCL-SLAM (w/o keyframe) would process every scan in LIO. Moreover, we calculate the average runtime per scan and total processing runtime of the whole sequence in other modules for better comparison since pose searching and map reconstruction are only triggered when loop closure is detected; their average run time is calculated through cases when they are executed.

TABLE IV. PROCESSING TIME ON SEQUENCE *PLAYGROUND*

	SCL-SLAM (w/ keyframe)	SCL-SLAM (w/o keyframe)
	Average (ms) / Total (s)	Average (ms) / Total (s)
LIO	5.95 / -	6.25 / -
Optimization	4.02 / 18.43	9.36 / 83.78
Pose Searching*	1.97 / 0.13	2.05 / 17.67
Map Recontruction*	0.64 / 0.04	0.34 / 2.92
Making Descriptors	0.07 / 0.34	0.41 / 3.64
Total	46.95 / 215.15	103.96 / 930.90

* The average run time is calculated through cases when they are executed.

The mean time consumption per LiDAR frame of SCL-SLAM (w/o keyframe) is 103.96 ms, and the total time is 930.90 s for the whole sequence, which exceeds the sequence duration. Therefore, without Keyframe selection, SCL-SLAM shows weak real-time performance. While SCL-SLAM (w/ keyframe) is only 46.95 ms per scan and 215.15 s for the complete processing, the run-time results show that our keyframe selection can make computation more efficient enough to achieve real-time performance.

E. Multi-robot Mapping Applications

As mentioned in Section I, SCL-SLAM can be extended to a centralized multi-robot SLAM framework. The LIO and back-end are deployed in different robots and the base station. The lightweight Scan Context descriptor is constructed in the robot and sent to the base station, permitting a data-efficient exchange among robots. The overview of our centralized multi-robot SLAM system is shown in Fig.6.

We collect the sequences of three geographically adjacent building groups to generate a multi-robot scenario, referred to *Building-1*, *Building-2* and *Building-3*. Each robot estimates

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on February 28,2023 at 09:52:19 UTC from IEEE Xplore. Restrictions apply.

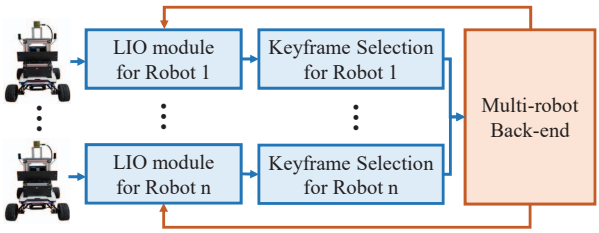
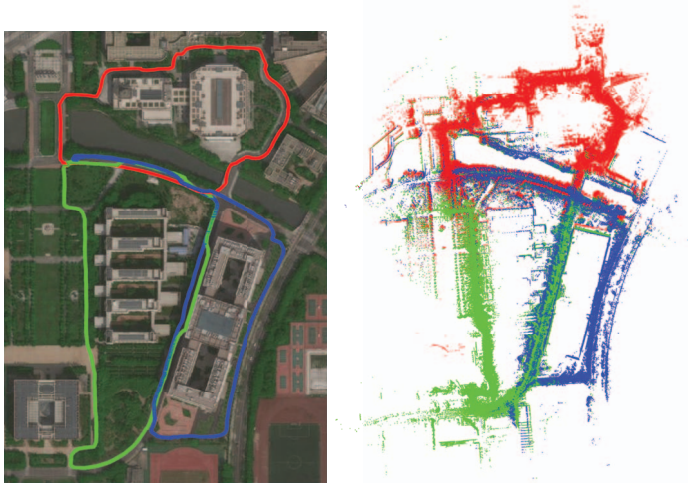


Fig. 6. Overview of the centralized multi-robot SLAM system architecture.

its trajectory onboard and sends Scan Context descriptors with relevant poses and keyframes to the base station. The base station collects collaborative information from three robots, maintains a globally consistent coordinate by place recognition, and performs centralized pose graph optimization to maintain a consistent global map. Furthermore, for higher accuracy of localization of each robot, The base station sends the global localization feedback to each robot to improve its accuracy and effectiveness.

The trajectory and constructed global map result of Sequence *Building-1*, *Building-2* and *Building-3* using our method is also shown in Fig.7. The red in the figure is for sequence *Building-1*, the blue is for sequence *Building-2*, and the green is for sequence *Building-3*.



(a) Multi-robot trajectory result (b) Multi-robot mapping result

Fig. 7. Multi-robot experiment result. The red is for robot 1, the green is for robot 2, and the blue is for robot 3.

IV. CONCLUSION

This work proposes a complete real-time SLAM framework by integrating the loop closure module with Scan Context descriptor into the direct LiDAR-Inertial odometry. The front-end produces the odometry and undistorted scans to the keyframe selection module, and the back-end detects loop closure with the selected scans. Then a pose graph is constructed with keyframe states and scan context loop to solve the MAP state estimation problem. Moreover, we evaluate the proposed system on both public LiDAR datasets and real-world experiments. Overall, our system is real-time capable and superior to the existing state-of-the-art LIO systems in terms of tracking and mapping accuracy. Finally, we extend our method to a centralized multi-robot SLAM framework and verify its collaborative ability to construct a global map through a real-world environment.

REFERENCES

[1] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
[2] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 3, pp. 611–625, 2017.
[3] T. Schops, T. Sattler, and M. Pollefeys, "Bad slam: Bundle adjusted direct rgb-d slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 134–144.
[4] C. Debeunne and D. Vivet, "A review of visual-lidar fusion based simultaneous localization and mapping," *Sensors*, vol. 20, no. 7, p. 2068, 2020.
[5] J. Zhang and S. Singh, "Loam: Lidar odometry and mapping in real-time." vol. 2, no. 9, pp. 1–9, 2014.
[6] T. Shan and B. Englot, "Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 4758–4765.
[7] J. Lin and F. Zhang, "Loam livox: A fast, robust, high-precision lidar odometry and mapping package for lidars of small fov," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 3126–3131.
[8] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, 2020, pp. 5135–5142.
[9] K. Li, M. Li, and U. D. Hanebeck, "Towards high-performance solid-state-lidar-inertial odometry and mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5167–5174, 2021.
[10] Z. Wang, L. Zhang, Y. Shen, and Y. Zhou, "D-liom: Tightly-coupled direct lidar-inertial odometry and mapping," *IEEE Transactions on Multimedia*, 2022.
[11] K. Chen, B. T. Lopez, A.-a. Agha-mohammadi, and A. Mehta, "Direct lidar odometry: Fast localization with dense point clouds," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2000–2007, 2022.
[12] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lio2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, 2022.
[13] A. E. Johnson and M. Hebert, "Using spin images for efficient object recognition in cluttered 3d scenes," *IEEE Transactions on pattern analysis and machine intelligence*, vol. 21, no. 5, pp. 433–449, 1999.
[14] R. B. Rusu, N. Blodow, Z. C. Marton, and M. Beetz, "Aligning point cloud views using persistent feature histograms," in *2008 IEEE/RSJ international conference on intelligent robots and systems*, 2008, pp. 3384–3391.
[15] S. Belongie, J. Malik, and J. Puzicha, "Shape context: A new descriptor for shape matching and object recognition," *Advances in neural information processing systems*, vol. 13, 2000.
[16] S. Salti, F. Tombari, and L. Di Stefano, "Shot: Unique signatures of histograms for surface and texture description," *Computer Vision and Image Understanding*, vol. 125, pp. 251–264, 2014.
[17] L. He, X. Wang, and H. Zhang, "M2dp: A novel 3d point cloud descriptor and its application in loop closure detection," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 231–237.
[18] R. Dubé, D. Dugas, E. Stumm, J. Nieto, R. Siegwart, and C. Cadena, "Segmatch: Segment based place recognition in 3d point clouds," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 5266–5272.
[19] G. Kim and A. Kim, "Scan context: Egocentric spatial descriptor for place recognition within 3d point cloud map," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 4802–4809.
[20] P. Agarwal, G. D. Tipaldi, L. Spinello, C. Stachniss, and W. Burgard, "Robust map optimization using dynamic covariance scaling," in *2013 IEEE International Conference on Robotics and Automation*, 2013, pp. 62–69.
[21] Y. Cai, W. Xu, and F. Zhang, "ikd-tree: An incremental kd tree for robotic applications," *arXiv preprint arXiv:2102.10808*, 2021.
[22] F. Dellaert, "Factor graphs and gtsam: A hands-on introduction," Georgia Institute of Technology, Tech. Rep., 2012.
[23] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert, "isam2: Incremental smoothing and mapping using the bayes tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012.
[24] D. Prokhorov, D. Zhukov, O. Barinova, K. Anton, and A. Vorontsova, "Measuring robustness of visual slam," in *2019 16th International Conference on Machine Vision Applications (MVA)*. IEEE, 2019, pp. 1–6.

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on February 28,2023 at 09:52:19 UTC from IEEE Xplore. Restrictions apply.